

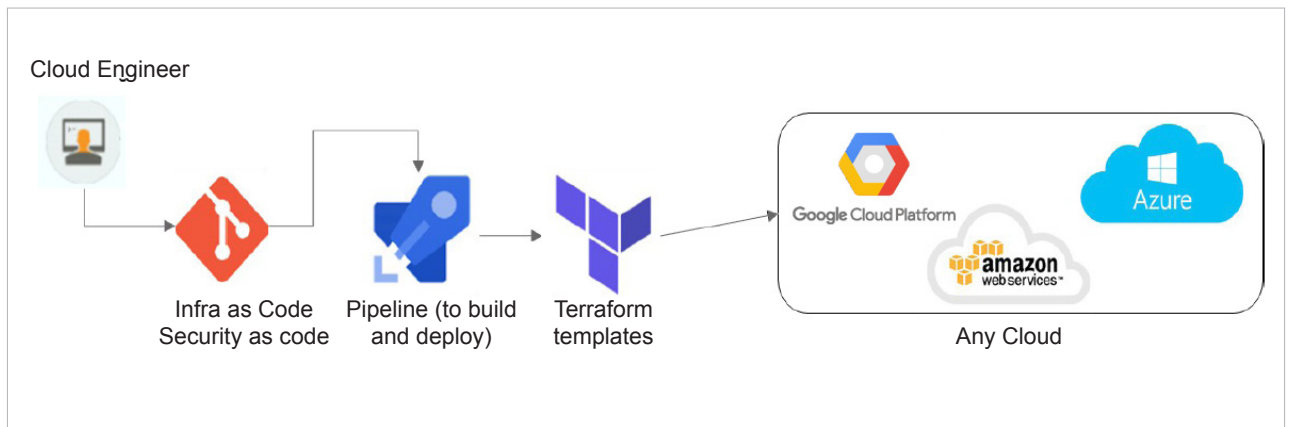


Figure 7: Terraform templates for cloud resource provisioning

- **CloudForms management engine appliance** is the backbone for the cloud infrastructure.
- **SmartProxy** service is the brain of the platform and sets a SmartProxy agent for carrying out the tasks triggered by the server.
- **CloudForms management engine server** is the heart of the platform and resides on the appliance to communicate between the SmartProxy layer and the virtual management service.
- **Virtual management database (VMDB)** contains the preconfiguration of the virtual infrastructure; it maintains the status of appliance tasks.
- **CloudForms management engine console** is the graphical interface or Web console; it has a dashboard on status.

Though Terraform is a very popular cloud management tool, Red Hat CloudForms provides great flexibility for private and public cloud management.

Infrastructure provisioning activities, during cloud adoption, are repetitive. Automating these helps to reduce the overall DevOps operational activities.

IaC is available as a native facility in Azure as ARM templates, in AWS as CloudFormation templates, and in GCP as Google deployment manager (GDM). A cloud agnostic solution for IaC for a multi-cloud environment is

Hashicorp's Terraform template, which can be JSON (to export/reuse from native scripts) or YAML script based. It can be integrated to any cloud service provider's native deployment pipeline or a third-party pipeline like Ansible to enable infrastructure provisioning activities on-the-go. Figure 7 highlights Terraform templates for cloud resource provisioning.

In large scale infrastructure creation, we need to build test instances and create pre-production or production instances during horizontal scaling. The task is easier using IaC with Terraform scripts, which can run with resource schedulers, and create network configurations or a secured environment setup for software defined networks (SDN) using Terraform templates.

Terraform scripts can be used to spin-up instances or destroy disposable temporary test environments in a cloud platform. Its templates can be validated before actual provisioning and hence Terraform workflow involves *Write* (create Terraform templates for IaC), *Plan* (preview the changes before actually applying them in the cloud environment to validate) and *Apply* (to actually provision infrastructure quickly).

For multi-cloud architecture, it is easy to adapt Terraform templates for IaC to quickly create a uniform setup and configure for multiple environments, reduce complexity in creating multiple parallel scripts, and reuse as much as possible between these scripts for each cloud service provider in the multi-cloud setup. **END** 🐧

References

- [1] <https://www.cloudops.com/>
- [2] <https://www.cloudbolt.io/best-cloud-management-platforms-2019/>
- [3] <https://www.manageiq.org/docs/guides/architecture>
- [4] <https://access.redhat.com/products/red-hat-cloudforms>
- [5] <https://blogs.gartner.com/marco-meinardi/2020/09/16/new-cloud-management-wheel-here/>

By: Dr Anand Nayyar and Dr Magesh Kasthuri

Dr Anand Nayyar is a PhD in wireless sensor networks and swarms intelligence. He works at Duy Tan University, Vietnam, and loves to explore open source technologies, IoT, cloud computing, deep learning and cyber security.

Dr Magesh Kasthuri is a senior distinguished member of the technical staff and principal consultant at Wipro Ltd. This article expresses his views and not that of Wipro.